

Summer School · EIV 2026

GNNs & LLMs: Reasoning Beyond Text

Dr. Daniel F. Pérez-Ramírez

Guest at Escuela Colombiana de Ingeniería Julio Garavito
2026-07-01

5 days · 24 hrs · In-Person



SECTION 0

Module 1 Recap

Module 1 Recap

P1. Aprendizaje supervisado es... (una opción correcta)

- a. Explorar e identificar la estructura de los datos o reducir la dimensión de los datos.
- b. Un agente que interactúa con un ambiente y obtiene una recompensa iterativamente
- c. Cuando tenemos una base de datos con muestras etiquetadas (labeled) y queremos aprender una función $f(x)=y$
- d. Monitorear el rendimiento del modelo en producción

Module 1 Recap

P2. ¿Cuáles de estos son ejemplos de datos i.i.d.? (3 de 4 opciones son correctas)

- a. Lanzar una moneda justa muchas veces y anotar cara o sello
- b. Medir la estatura de personas elegidas al azar de una población grande
- c. Tirar un dado varias veces y registrar el número que sale
- d. Los precios de cierre diarios de una acción en la bolsa durante un año

Module 1 Recap

P3 Los modelos de deep learning son modelos no paramétricos.

- a.** Verdadero
- b.** Falso

Module 1 Recap

P4 ¿Cuál es la diferencia entre el conjunto de test y el de validation al hacer el split de la base de datos? (una opción correcta)

- a.** El validation set sirve para ajustar los hiperparámetros del modelo, mientras que el test set es un conjunto reservado (hold-out) que no se usa durante el entrenamiento y sirve para estimar el desempeño final
- b.** El test set se usa para entrenar el modelo y el validation set para ajustar los weights (parámetros)
- c.** No hay diferencia; son dos nombres para el mismo conjunto
- d.** El validation set siempre debe ser más grande que el training set

Module 1 Recap

P5 ¿Cuál es la diferencia entre la covarianza y el coeficiente de correlación? (una opción correcta)

- a.** La covarianza siempre es positiva y la correlación puede ser negativa
- b.** La covarianza depende de la escala de las variables, mientras que el coeficiente de correlación está normalizado y acotado entre -1 y $+1$
- c.** Son exactamente lo mismo, solo cambia el nombre
- d.** El coeficiente de correlación depende de las unidades de las variables y la covarianza no

Module 1 Recap

P6 Un modelo que hace overfitting... (una opción correcta)

- a. Es demasiado simple y tiene error alto tanto en entrenamiento como en validación
- b. Memoriza el ruido de los datos de entrenamiento y falla al generalizar a datos nuevos
- c. Siempre tiene mejor desempeño en datos nuevos que en entrenamiento
- d. No puede entrenarse con gradient descent

Module 1 Recap

P7 ¿Cuál es el papel del learning rate (tasa de aprendizaje) en gradient descent? (una opción correcta)

- a.** Controla el tamaño del paso que se da en la dirección del gradiente negativo
- b.** Determina cuántas capas tiene la red neuronal
- c.** Es la etiqueta (label) que queremos predecir
- d.** Mide qué tan bien generaliza el modelo

Module 1 Recap

P8 En detección de fraude, donde el 99% de las transacciones son legítimas, ¿por qué la accuracy puede ser engañosa? (una opción correcta)

- a. Porque la accuracy siempre da 100% en estos casos
- b. Porque el fraude no se puede predecir con machine learning
- c. Porque la accuracy no se puede calcular con datos desbalanceados
- d. Porque un modelo que siempre predice "no fraude" alcanza 99% de accuracy pero no detecta ningún fraude

Course Overview

I · 5-6 hrs

ML Foundations

ML best practices. Train first ML model.

II · 5-6 hrs

LLMs + RAG

Fundamentals of LLMs and first RAG system.

III · 5-6 hrs

GNNs

Encode relational structure.

IV · 5-6 hrs

LLMs + GNNs

Combine all three end-to-end

V · 3-4 hrs

Wrap-up + Present

Demo your course project.

From Module I to Module II

An LLM is the analogous training loop you already know but scaled to billions of parameters.



FEATURES

Tokens — sub-word units of text

LABEL

The next token in the sequence

SCALE

Billions of parameters, internet-sized data

SECTION 1

What Are LLMs & How Do They “Learn Language”?

Language as a Prediction Task

“The cat sat on the **mat**”

“The Eiffel Tower is in **Paris**”

Given a sequence of words, predict the missing one.

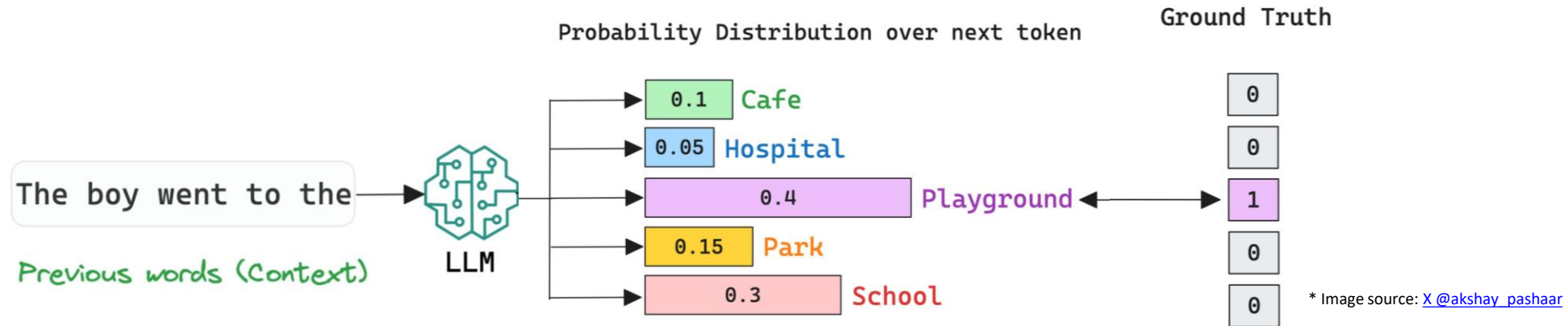
“The **Eiffel** Tower is in Paris”

Train this on all of the internet, and the model is forced to learn:

- **Grammar** — to make predictions parse
- **Facts** — Paris is in France
- **Reasoning patterns** — cause and effect

Masked Language Prediction

Language as a Prediction Task



$P(\text{'Playground'} \mid \text{'The boy went to the'}) = 0.4$
 $P(\text{'Stock Exchange'} \mid \text{'The boy went to the'}) = 0.01$
 $P(\text{'School'} \mid \text{'The boy went to the'}) = 0.3$
 $P(\text{'Tax Office'} \mid \text{'The boy went to the'}) = 0.02$

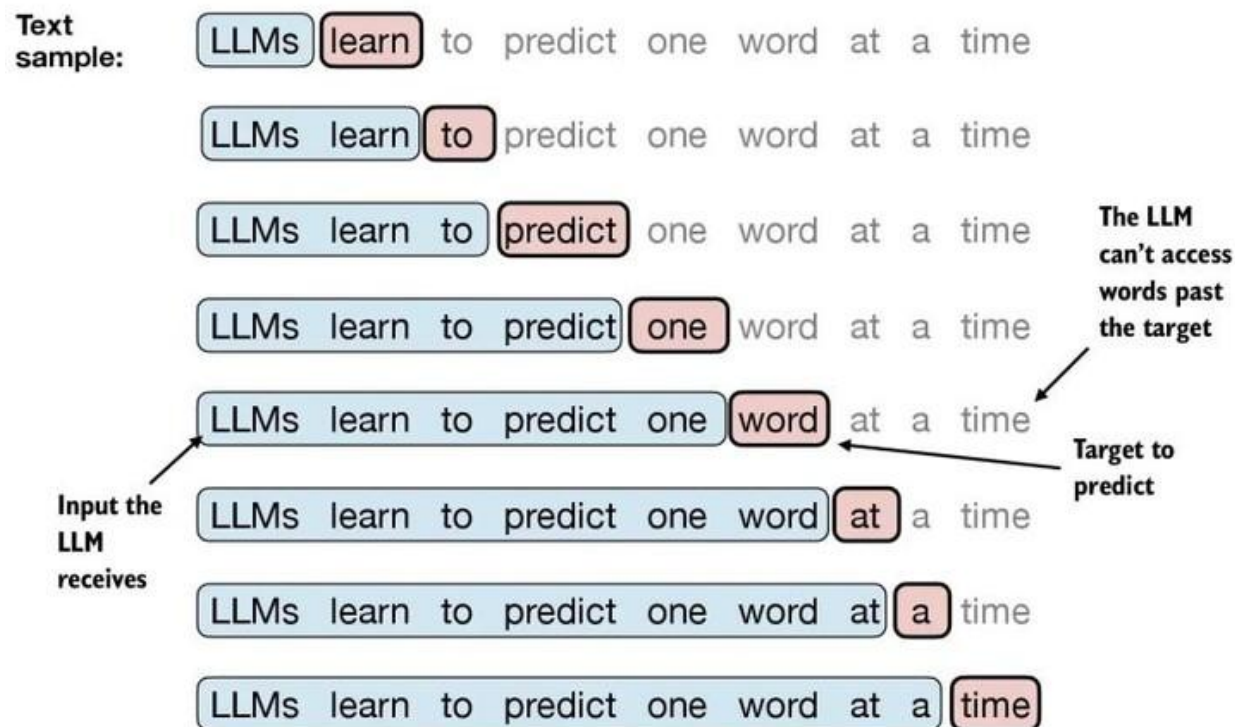
Vocabulary

Forms a probability distribution over the vocabulary size

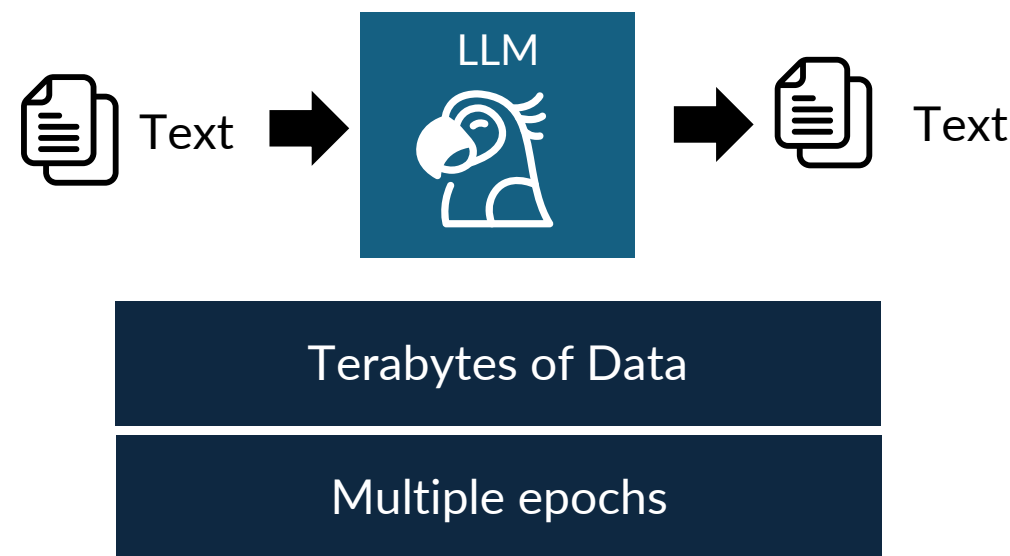
$$P(y_{t+1} \mid x_t, x_{t-1}, \dots, x_1, x_0)$$

$$\sum_y P(y_{t+1} \mid x_t \dots) = 1$$

Page 10 of 10



© 2024 Sebastian Raschka



Human: approx. 5700 years non-stop
reading (one time!)

What pre-training teaches — and what it doesn't

WHAT IT LEARNS

Syntax

Grammatical structure of language

Factual knowledge

World facts seen during training

Reasoning shortcuts

Common inference patterns

WHAT IT DOES NOT LEARN

Truth

No signal separates fact from plausible fiction

Causality

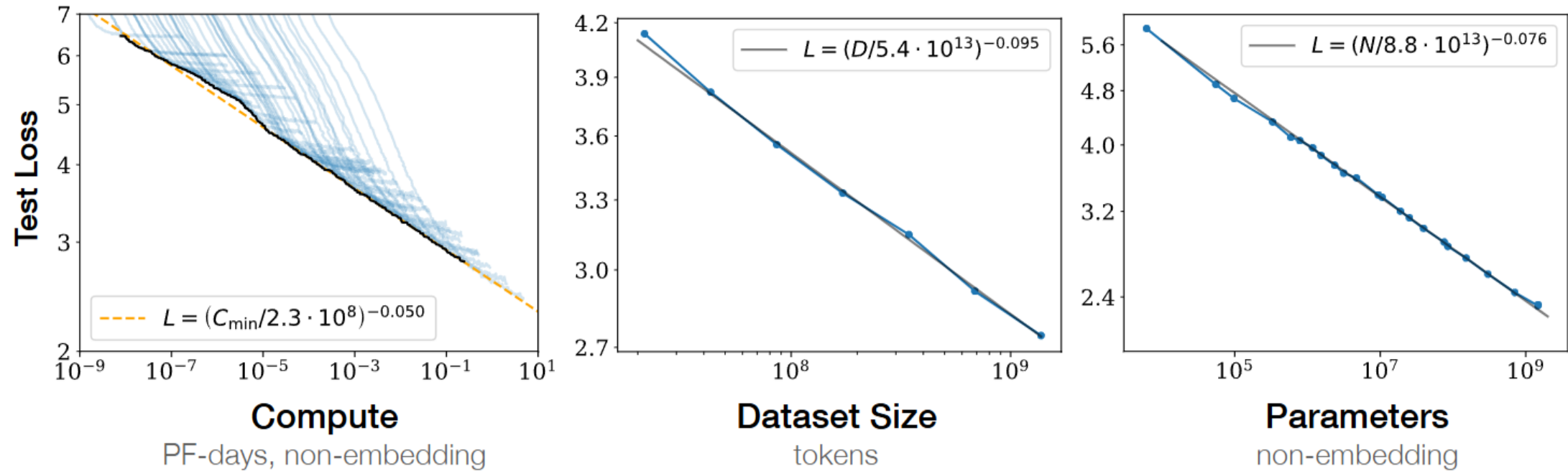
Correlation $\neq$ causation

Current state of the world

Training data has a cutoff date

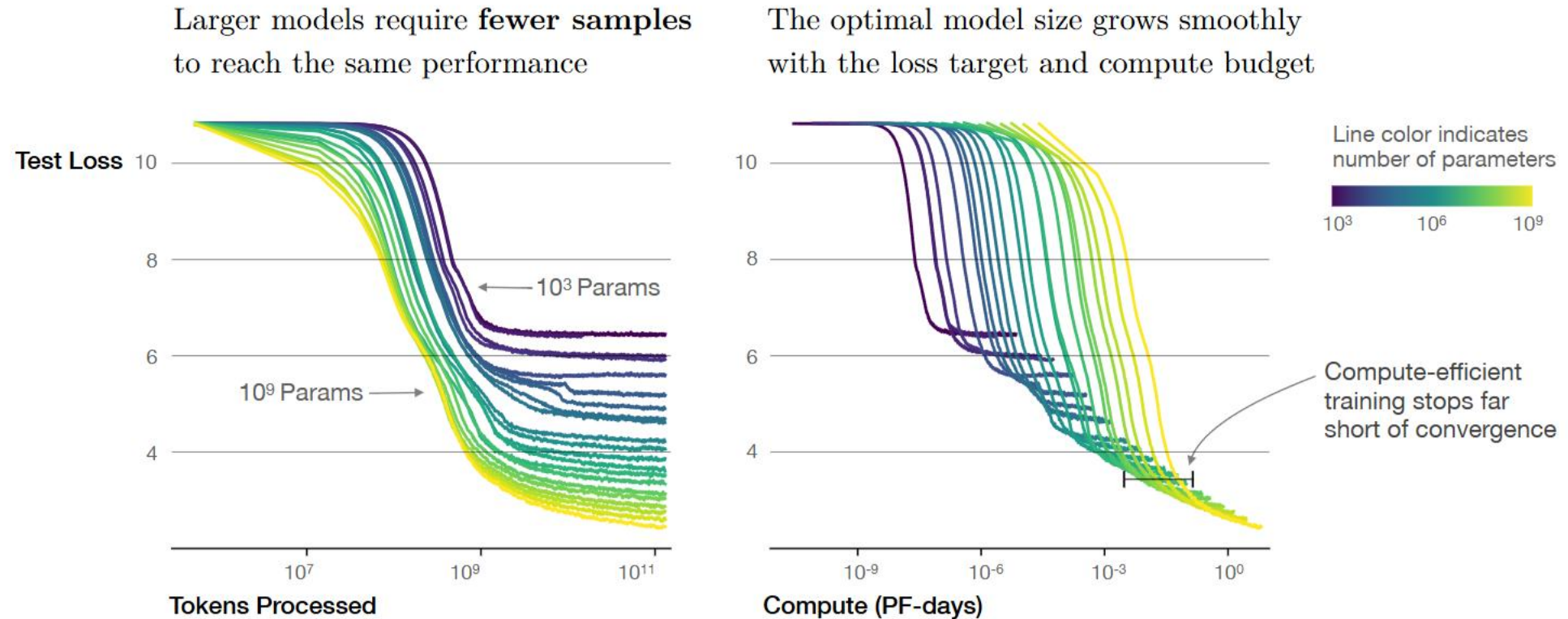
Scaling Laws for Neural Language Models

SCALING LAW · Kaplan et al. 2020

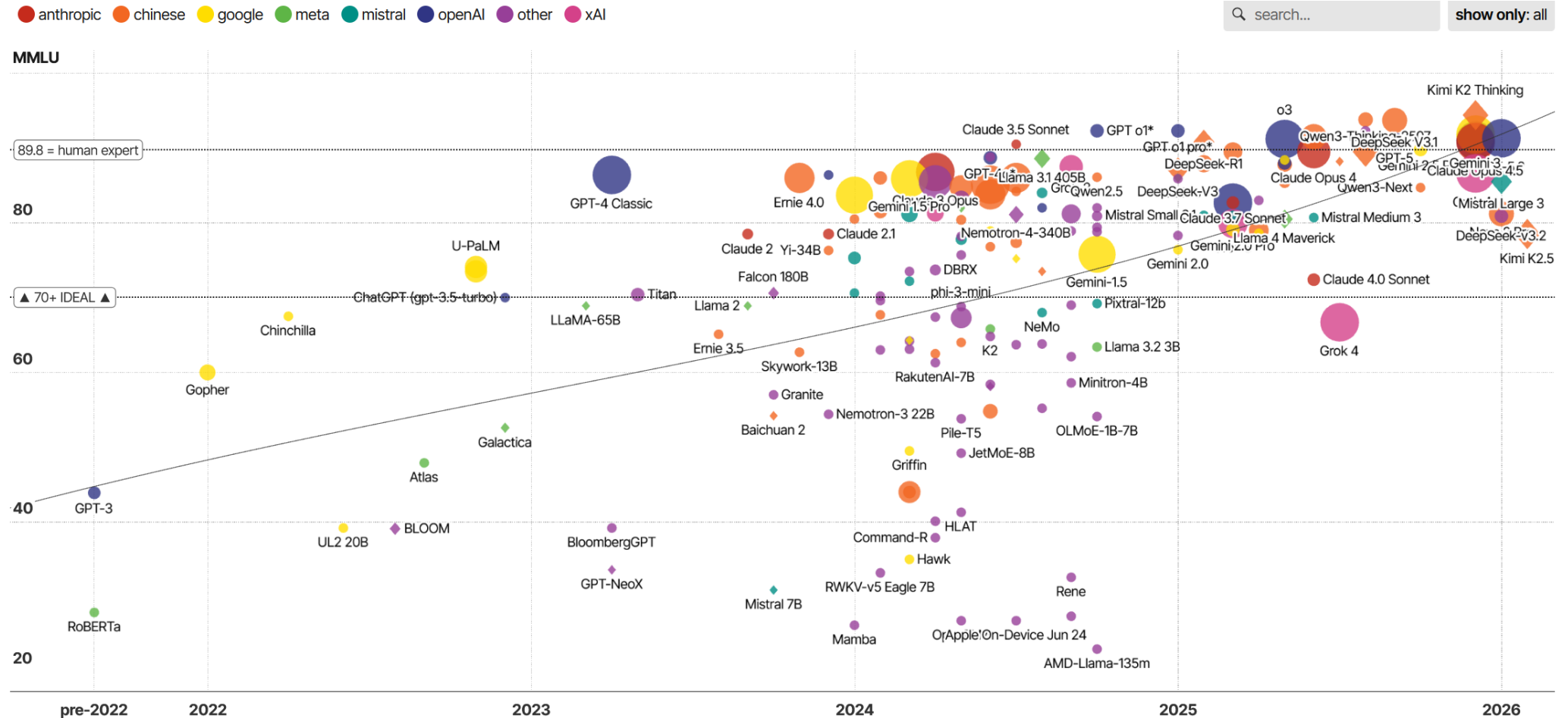


Scaling Laws for Neural Language Models

SCALING LAW · Kaplan et al. 2020



Landscape of Popular LLMs



SECTION 2

Foundations and Attention

Challenges with Language Processing

Length mismatch

Input and output
length may differ
Not known in advance

ESP

El | gato | corre

SWE

Katten | springer

Reordering

Word order differs
across languages

ESP

Ayer | **comimos** | en | el | restaurante

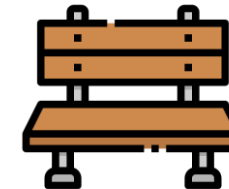
DEU

Gestern | haben | wir | im | Restaurant | **gegessen**

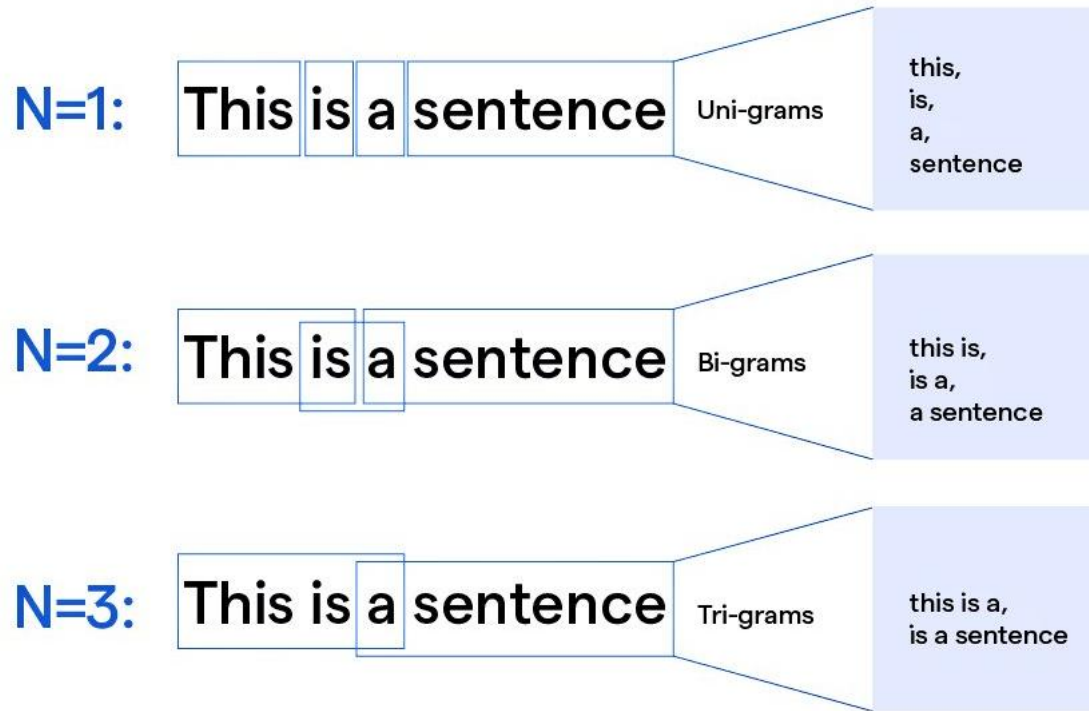
No 1-to-1 correspondence

Words may have
difference meaning

Banco



The Grandparent: n-gram models



Product rule factorizes whole sentence t

$$P(w_1 \dots w_t) = \prod_t P(w_t | w_1 \dots w_{t-1})$$



Assume is only depends on $n-1$ previous words

$$P(w_t | w_1 \dots w_{t-1}) \approx P(w_t | w_{t-n+1} \dots w_{t-1})$$



Estimate each conditional by maximum likelihood

$$P(w_t | w_{t-1}) = \text{count}(w_{t-1}, w_t) / \text{count}(w_{t-1})$$

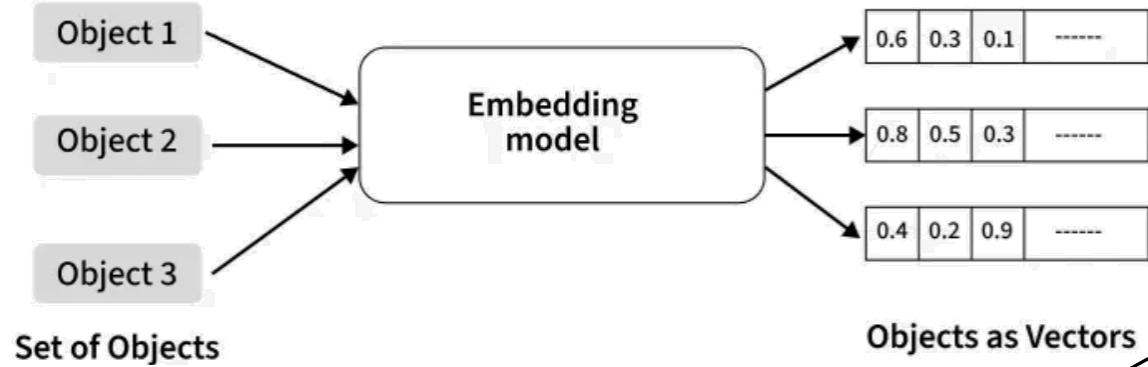
No generalization across similar words.

Limited Context and Sparsity.

High storage and processing cost.

The Principle: Word Embeddings

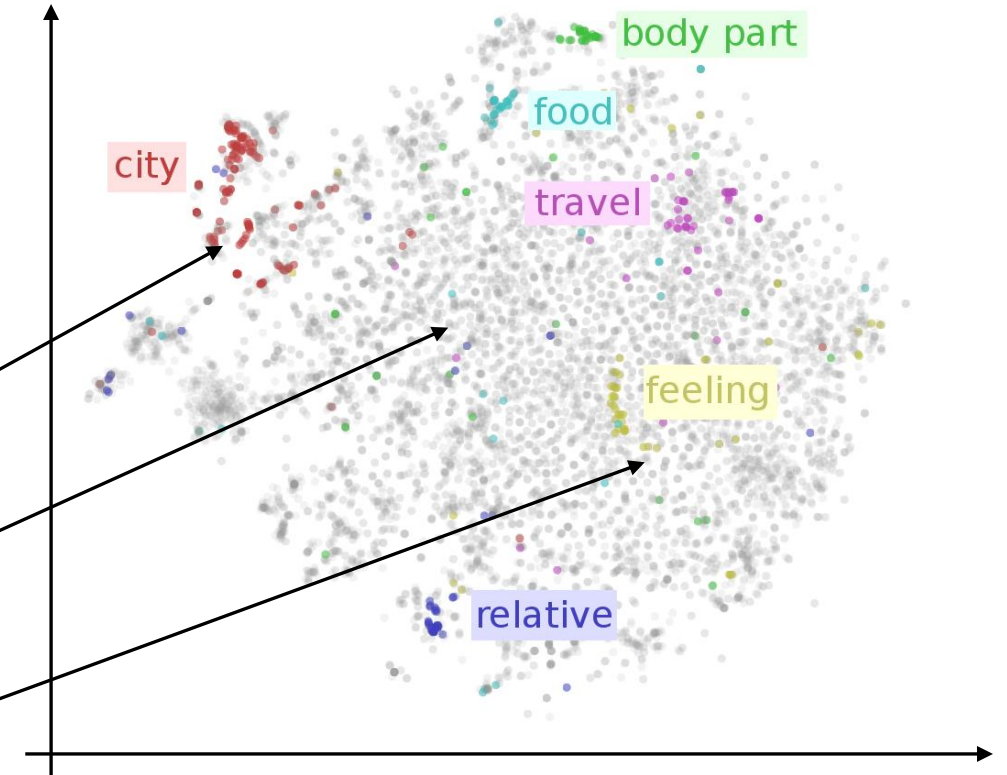
Represent a set of objects as numerical real-valued vectors.



Paris

Dog

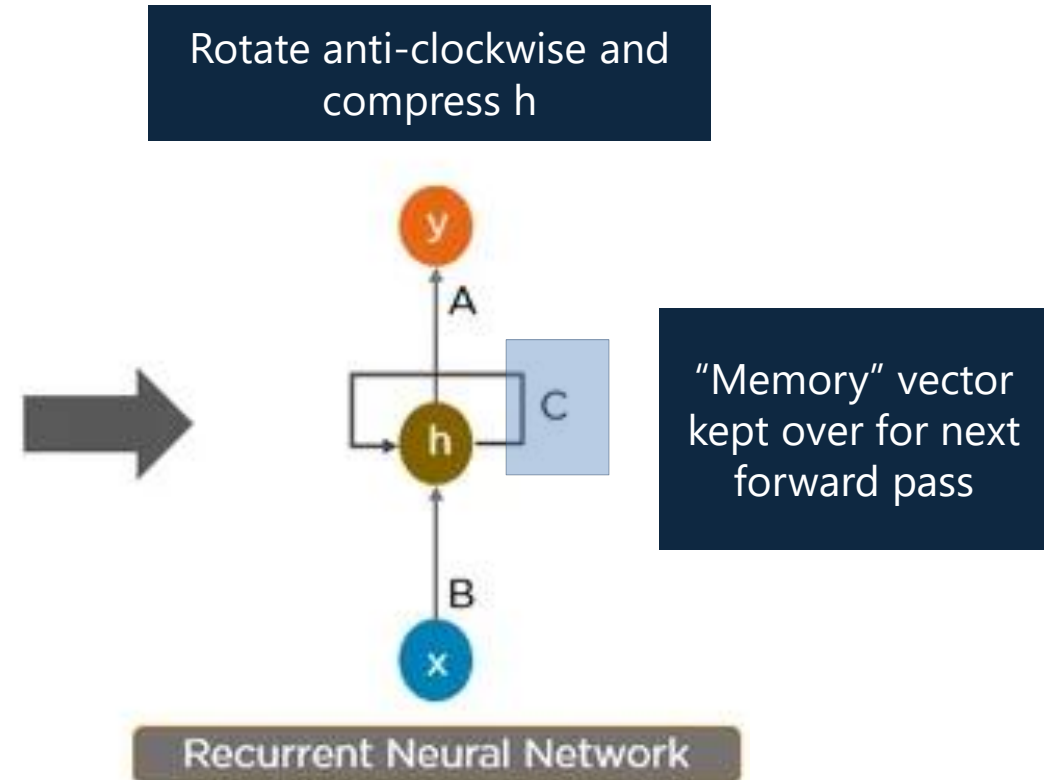
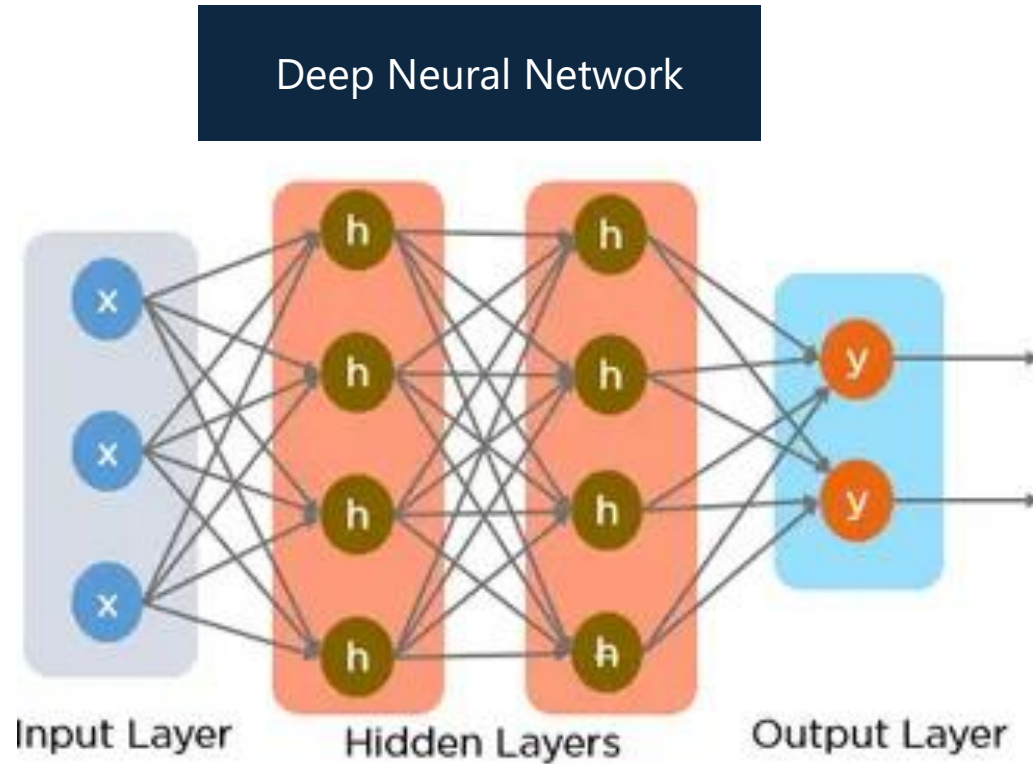
Happy



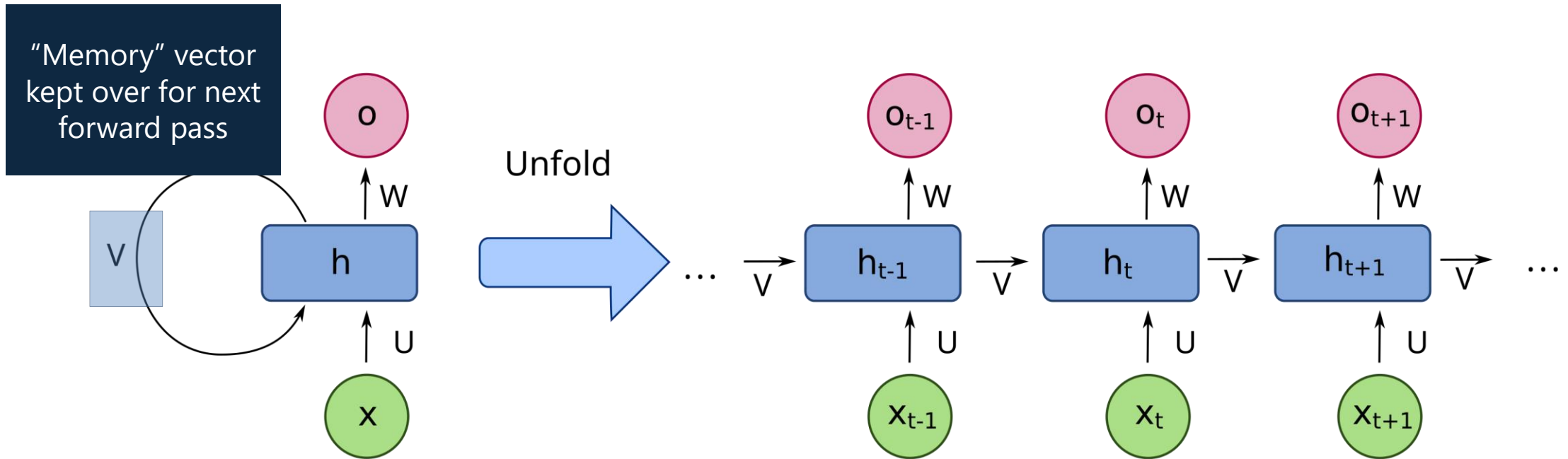
Bengio et al. (2003) — *A Neural Probabilistic Language Model*

Mikolov et al. (2013) — word2vec

Deep Learning for Sequences

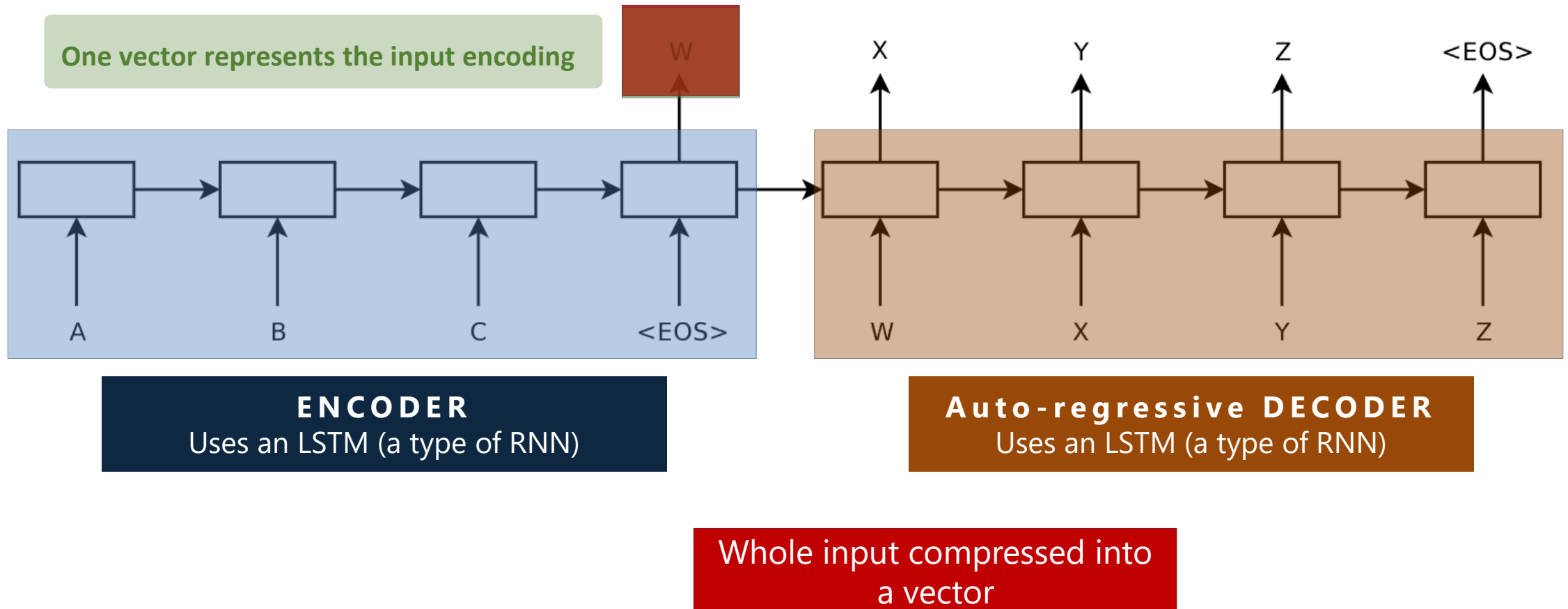


Deep Learning for Sequences



seq2seq

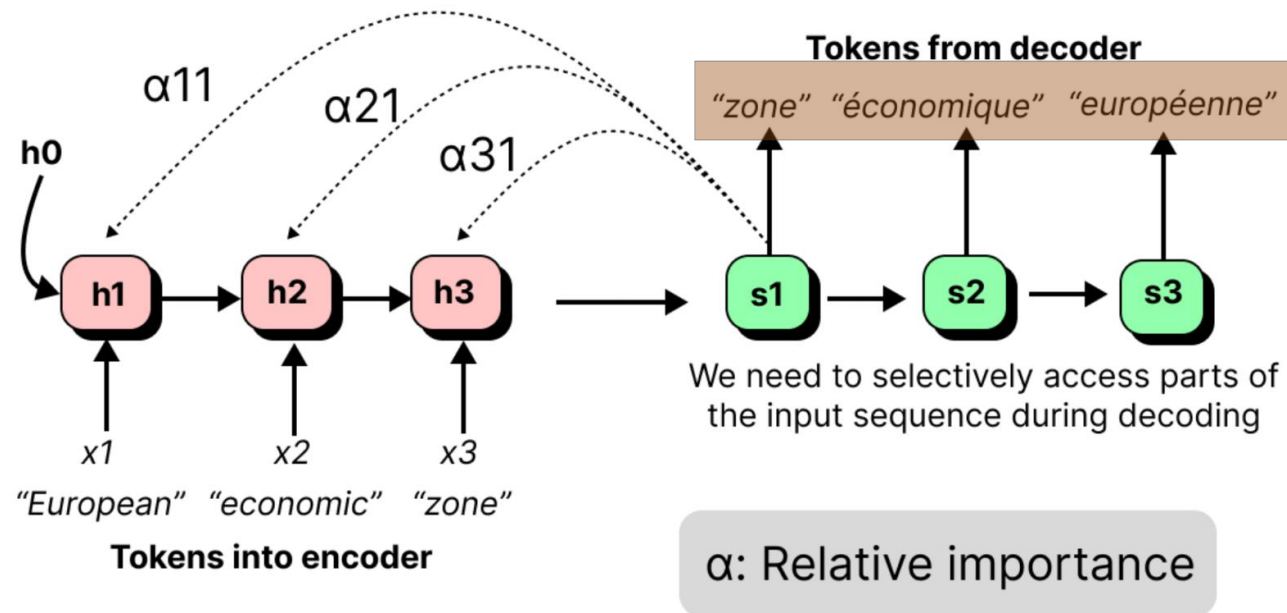
Sutskever, Vinyals & Le (2014) — *Sequence to Sequence Learning with Neural Networks*



Attention

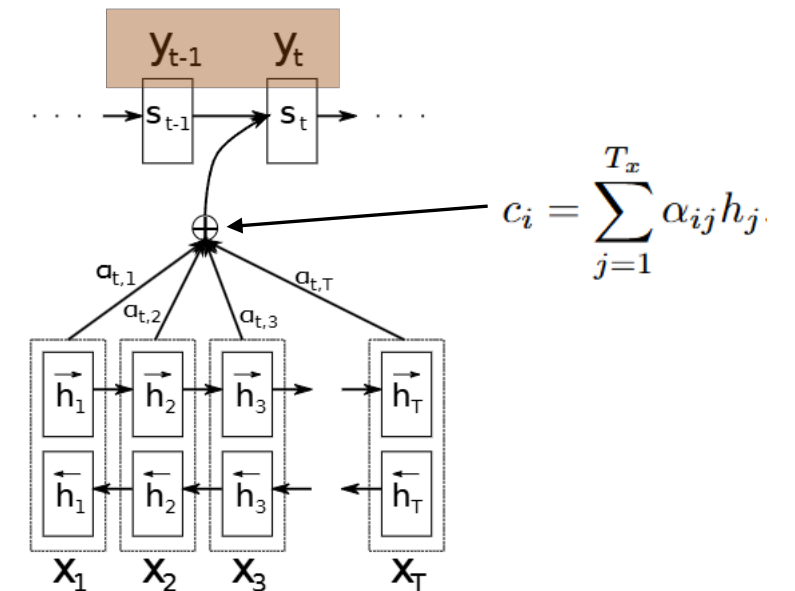
Bahdanau, Cho & Bengio (2014), "Neural Machine Translation by Jointly Learning to Align and Translate."

Let the output side (decoder) look back at the input side (hidden states of input).



Still uses a **RNN** (attention is bolted into it)

Quantify importance to each input h

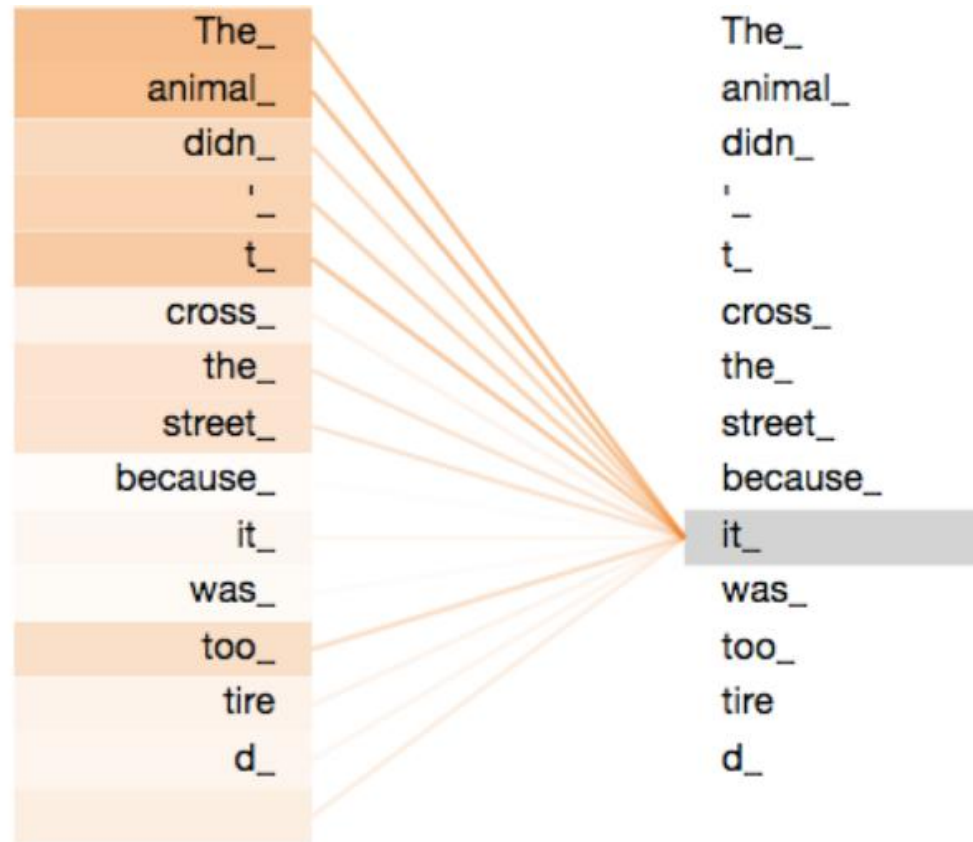


SECTION 3

Transformer Architecture

Attention

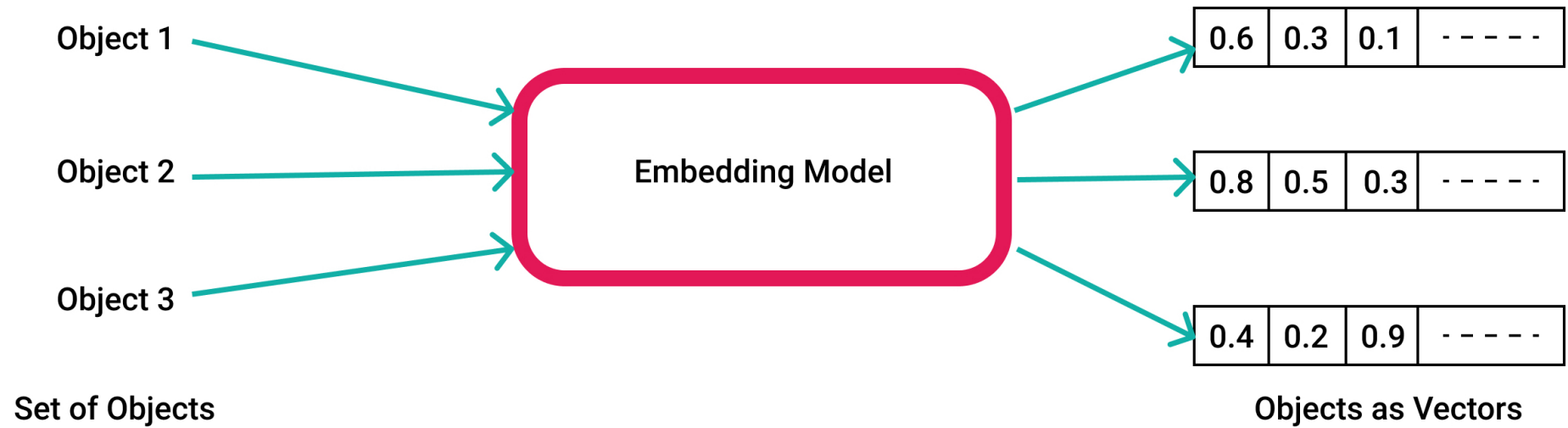
Allow new tokens to “attend” to previous tokens



"It" should "attend" to "the animal"

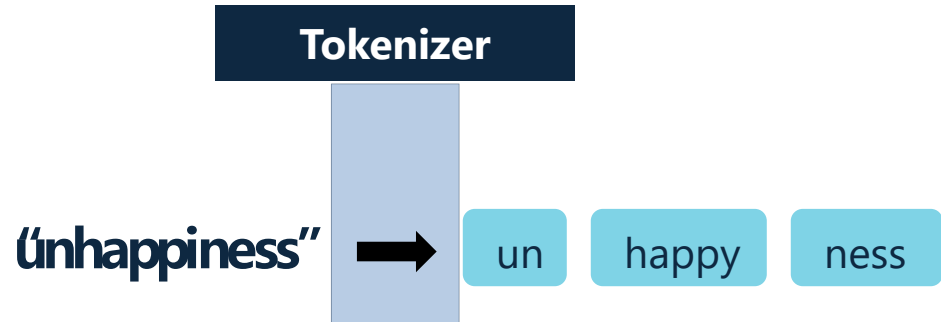
Embedding

Transform an object into a vector space



Vector Space

Tokens — the unit of language



Split into **sub-word** tokens — not words, not characters.

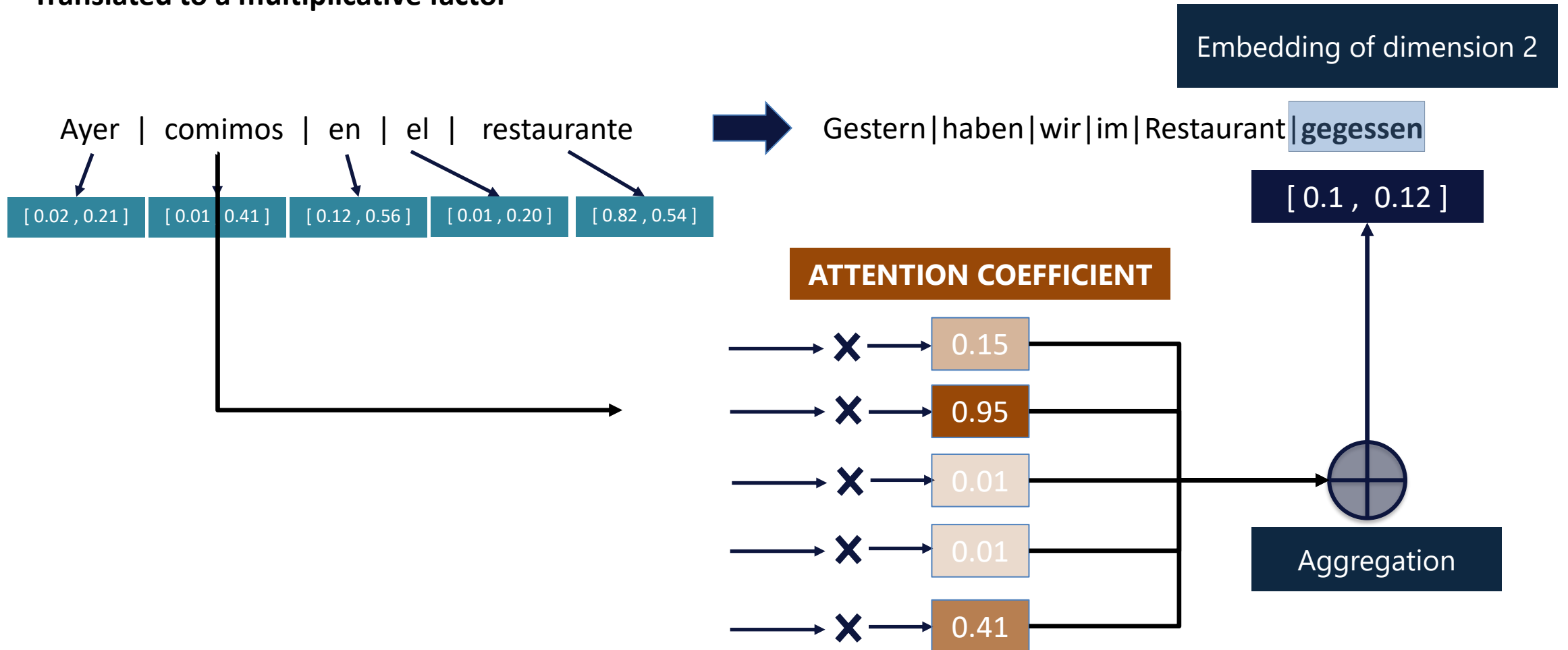
Typical vocabulary: **30 000–100 000** tokens.

Each token → an embedding vector

A point in high-dimensional space (e.g. 4096 dims). The Transformer operates on sequences of these vectors — let's build that intuition next.

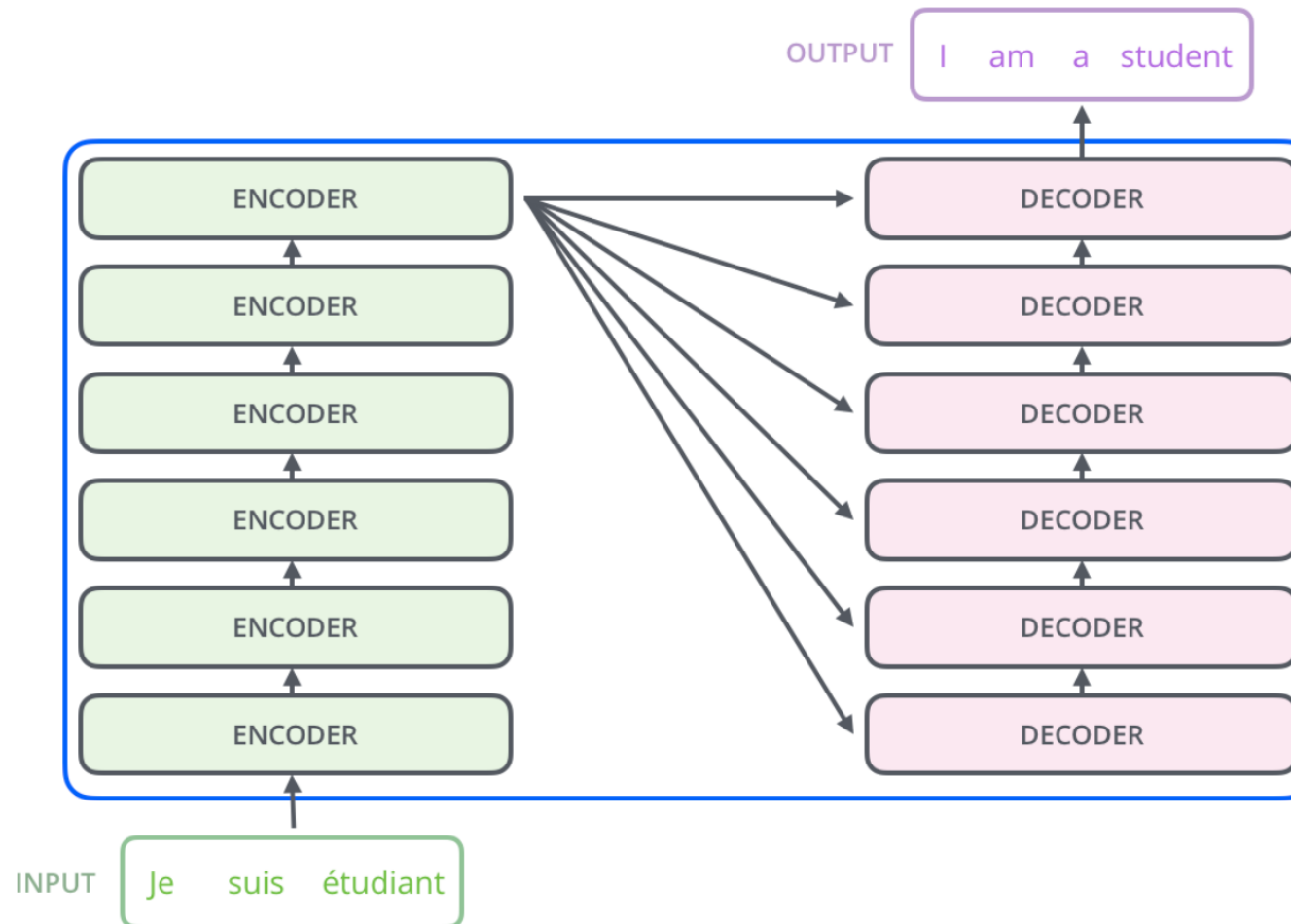
Attention in embedding space

Translated to a multiplicative factor



Attention is All you Need

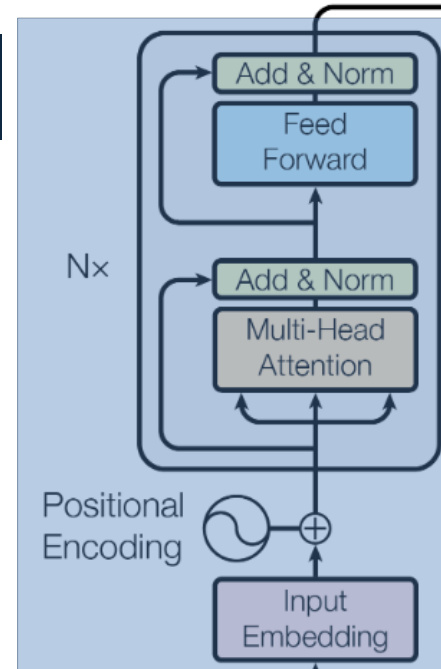
Ditch the RNN, and let attention do all the work of context processing



Images source. Jay Alamar. The Illustrated Transformer. <https://jalamar.github.io/illustrated-transformer/>

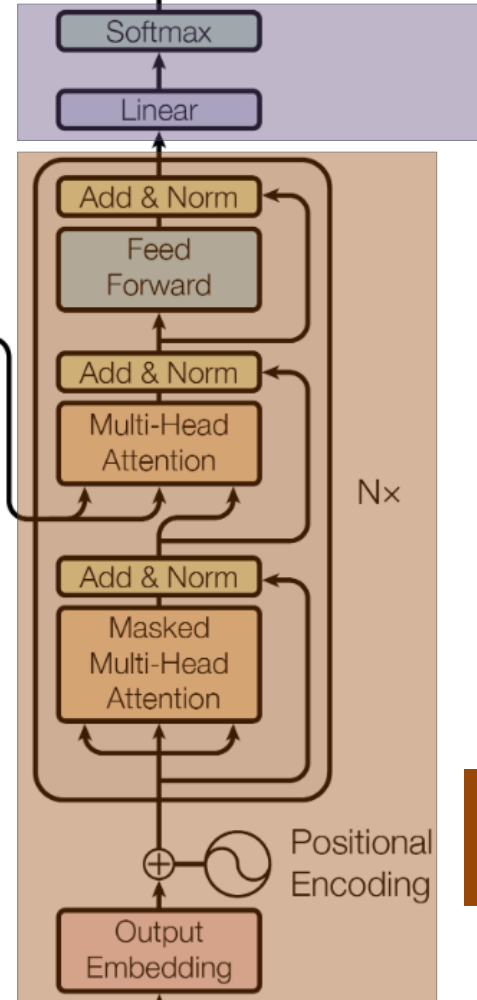
Transformer

ENCODER



Positional
Encoding

Inputs



Positional
Encoding

Outputs
(shifted right)

Output
Probabilities

Softmax

Linear

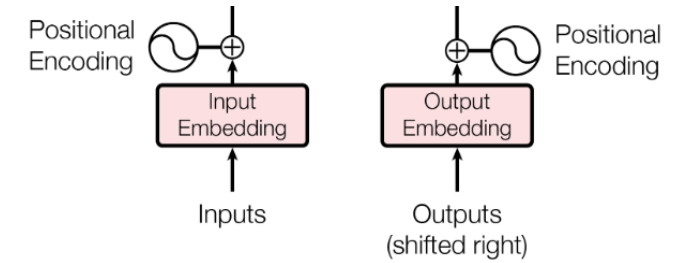
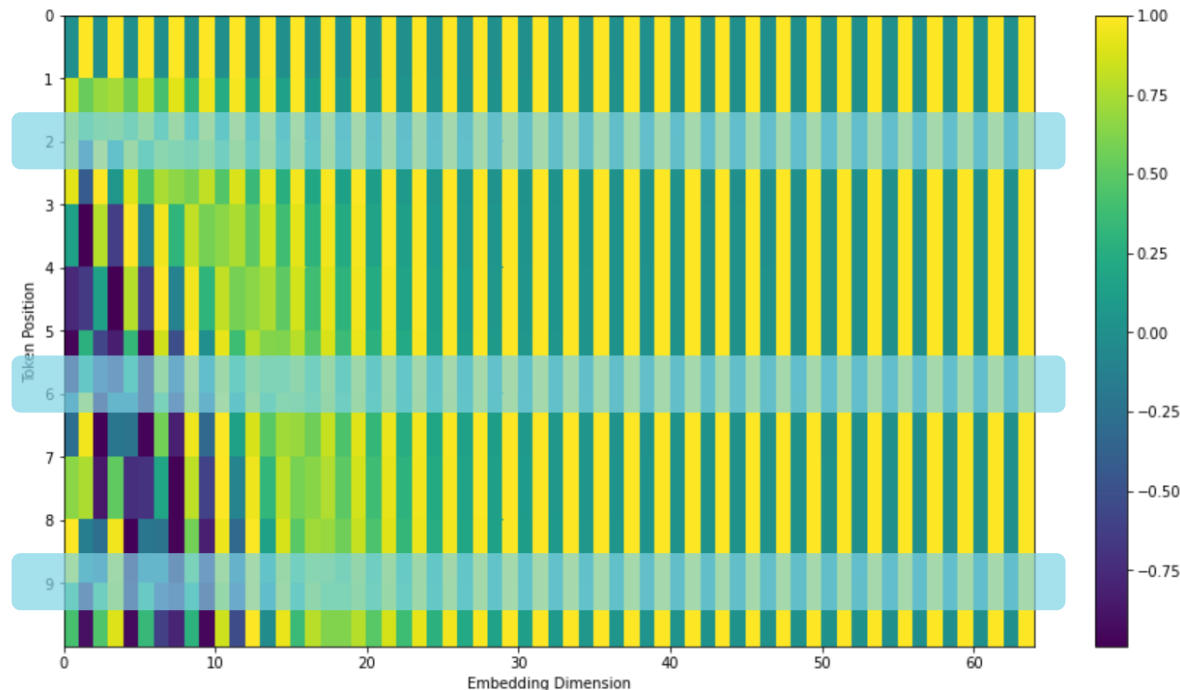
**Map back to probability distribution
over token vocabulary**

**Auto-regressive
DECODER**

Positional Encoding

Provide the model with a sense of **order** in which the tokens are in the sequence.
Understand the absolute and/or relative position of the tokens in the sequence.

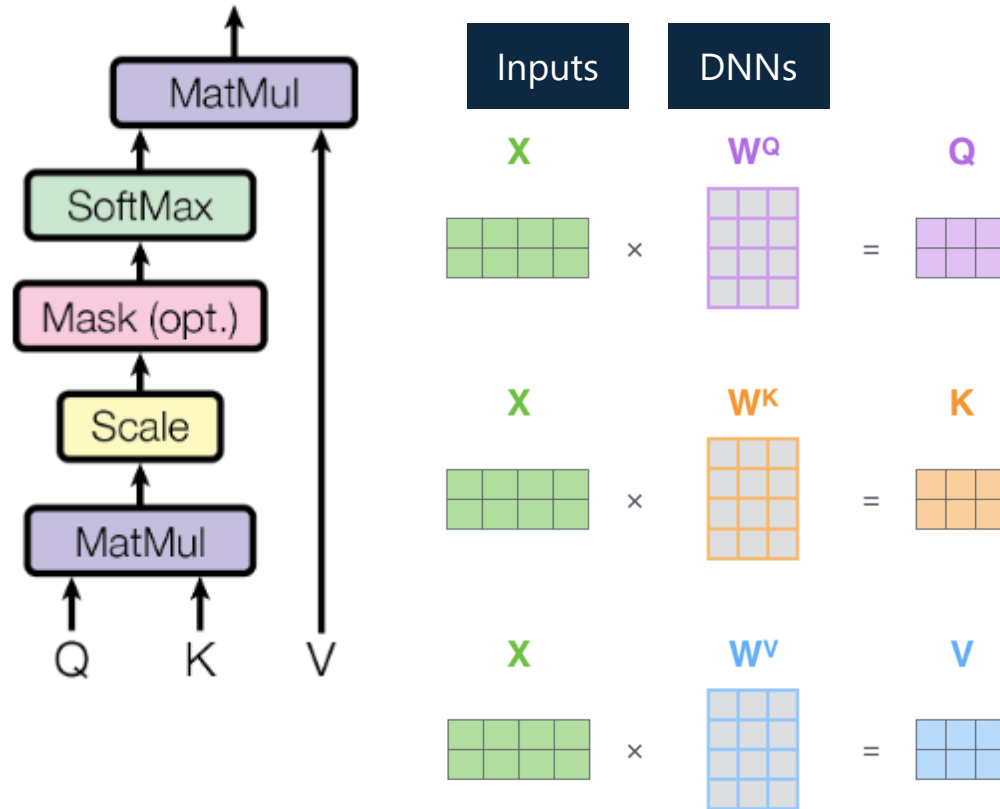
POSITION: 0 1 **2** ... **6** ... **9**
John likes coffee. He will get coffee from the coffee house.



The embedding would generate the same vector for "coffee"

PosEnc aids the model in differentiating these three

Scaled Dot-Product Attention



Query (Q)

What I'm looking for (new input x)

Key (K)

What each (previous) token is about.

Value (V)

The "content" to retrieve for that token.

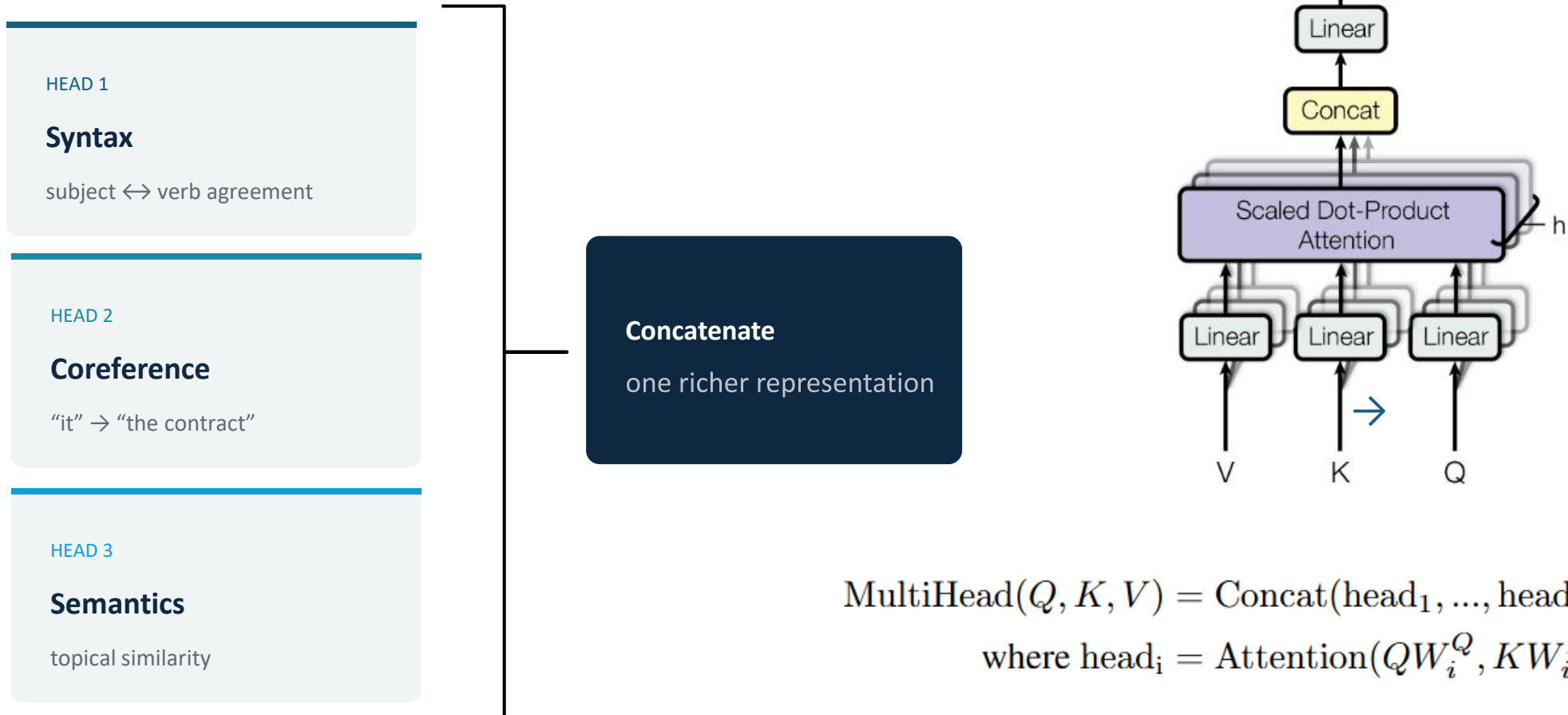


Match Q against each K → a high score shines the spotlight on that Value; a low score ignores it.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

A universal idea — it applies to Graph Attention Networks (Module III).

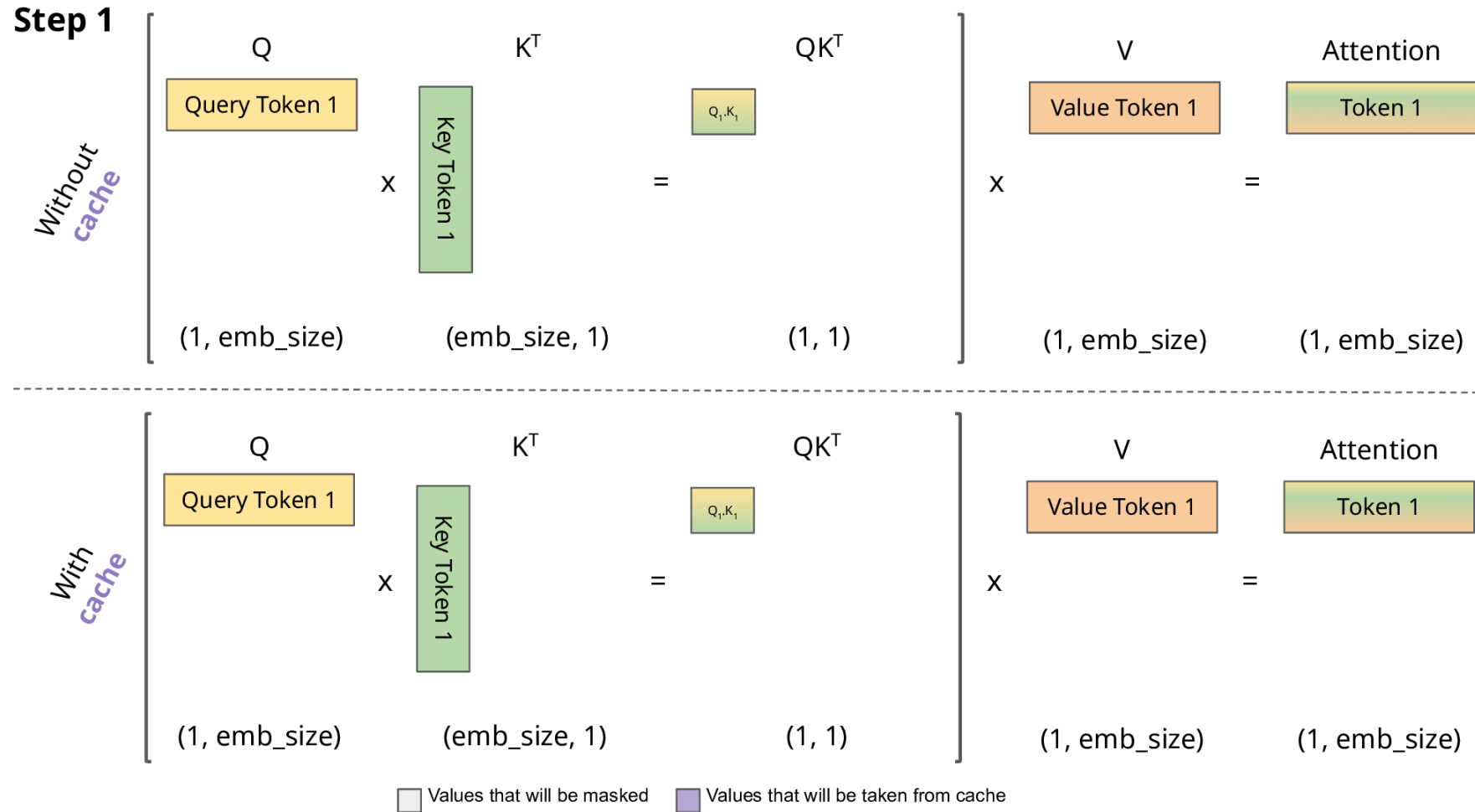
Multi-Head Attention



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

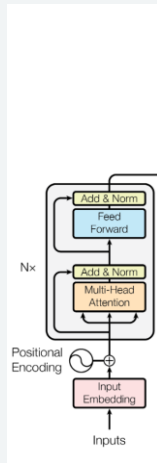
KV-Cache



Types of Transformers for NLP

Encoder BERT-style

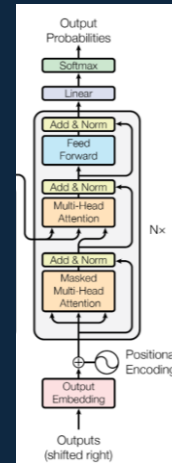
Sees the full sequence bidirectionally
→ understanding.



classification · embeddings · RAG retrieval

Decoder GPT-style

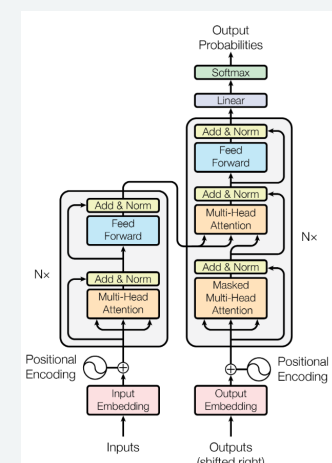
Sees only past tokens (causal) →
generation.



chat · code completion · creative text

Encoder-Decoder T5 / BART

Reads input, generates output →
transformation.



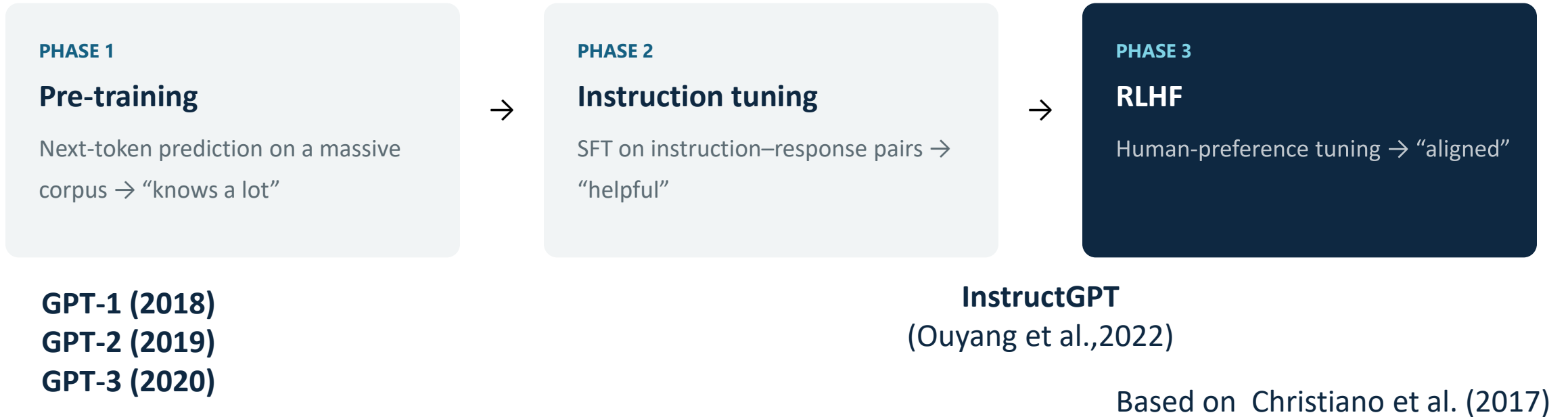
translation · summarisation

Course focus: decoder (Ollama LLMs) + encoder (sentence-transformers for RAG).

SECTION 4

LLM Training Phases

From raw model to helpful assistant

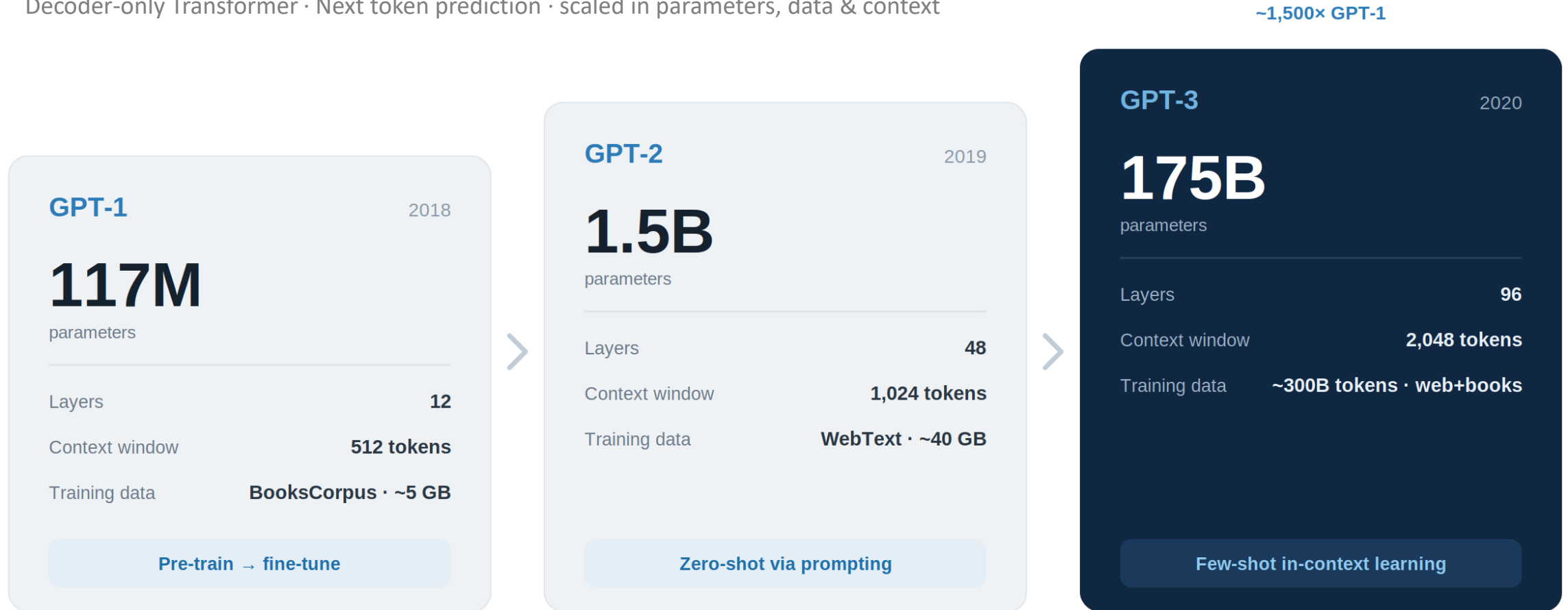


None of these phases mitigates hallucinations

Phase 1: Pre-Training and instruction tuning

Same recipe, ~1500× bigger: GPT-1 → GPT-2 → GPT-3

Decoder-only Transformer · Next token prediction · scaled in parameters, data & context



Phase 2: Supervised Fine-Tuning

InstructGPT

(Ouyang et al., 2022),
"Training language models
to follow instructions with
human feedback."

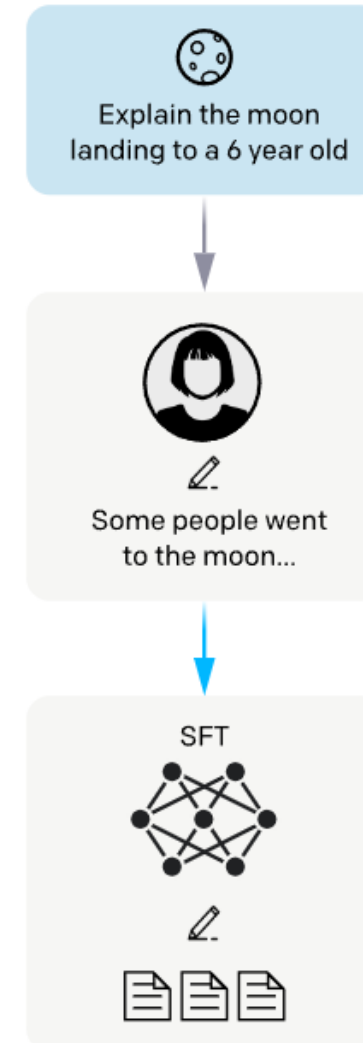
Collect demonstration data
and train a supervised policy

Becomes a "helpful assistant."

A prompt is
sampled from our
prompt dataset.

A labeler
demonstrates the
desired output
behavior.

This data is used
to fine-tune GPT-3
with supervised
learning.



Phase 3: Reinforcement Learning from Human Feedback (RLHF)

InstructGPT

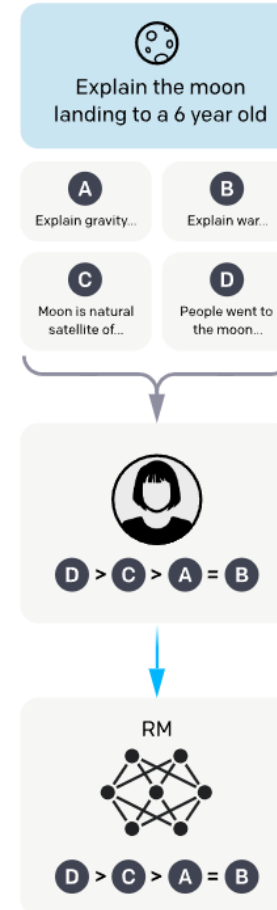
(Ouyang et al., 2022),
"Training language models to follow instructions with human feedback."

Assistant becomes "customer"- friendly

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.

This data is used to train our reward model.

Step 3

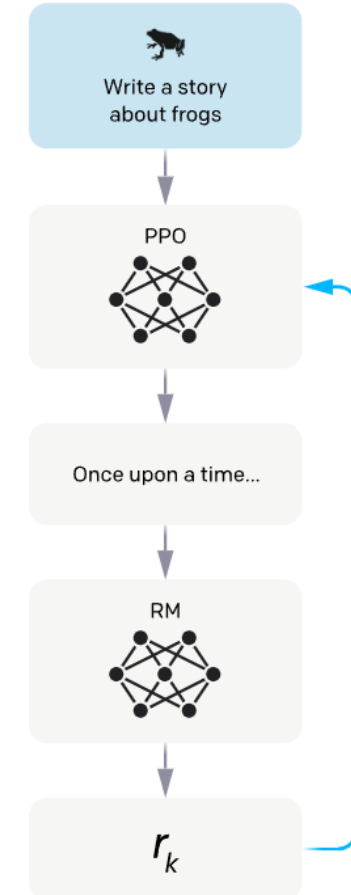
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Open-source vs. proprietary

PROPRIETARY · API-ONLY

GPT-4o · Claude · Gemini

State of the art — but costly, opaque, internet-dependent.

OPEN-SOURCE · RUN LOCALLY

Llama 3 · Mistral · Qwen · Phi-3 · SmolLM2

Reproducible, free, no API key — runs on your laptop.

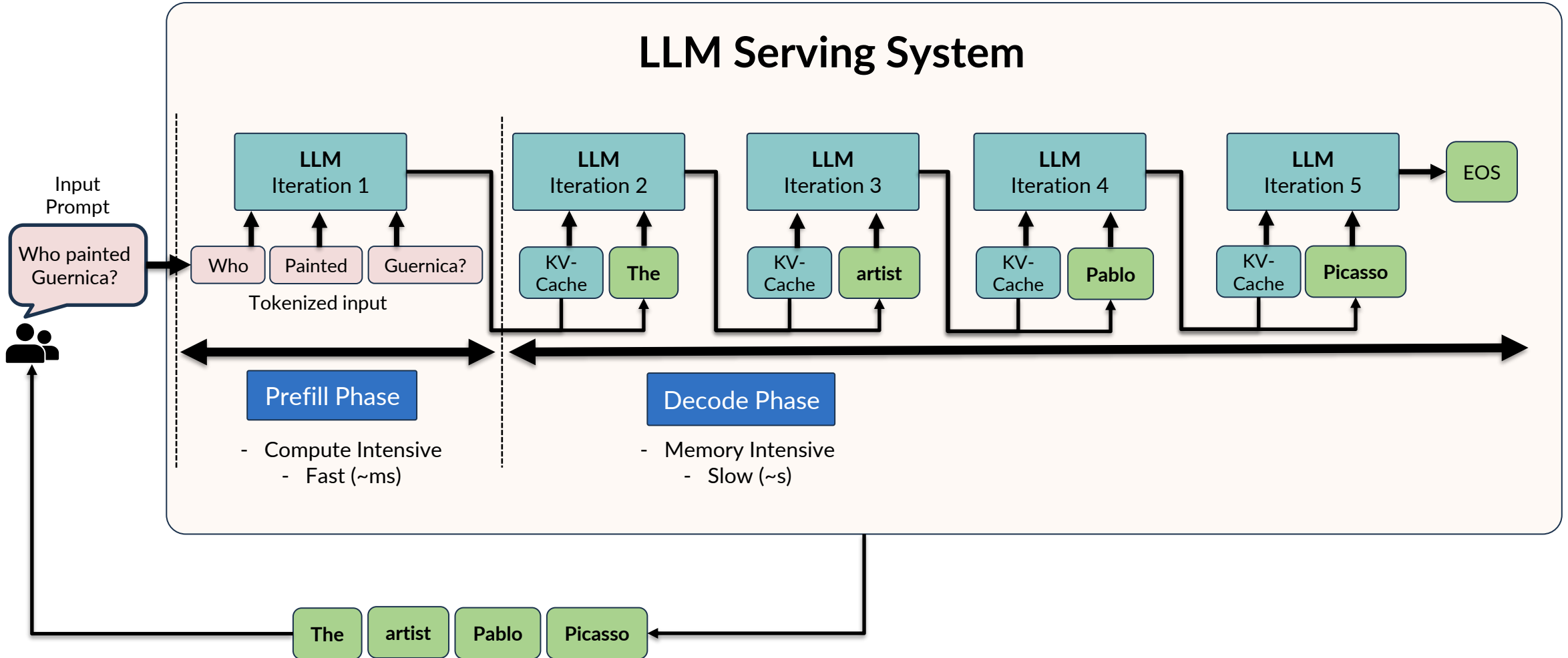
```
$ ollama run llama3.2
```

Course uses open models via Ollama · Colab fallback: SmolLM2-1.7B-Instruct.

SECTION 5

Inference and Computational Cost

LLMs in Production Systems



How to Choose the next token?

Greedy

Select the highest

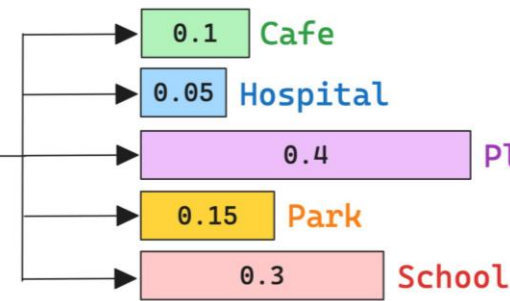
The boy went to the

Previous words (Context)



LLM

Probability Distribution over next token



Ground Truth

0
0
1
0
0

Top-k

Select the k highest, then sample uniformly

The dog is

[Top-k]	[Token]	[Top-p (p: Probability)]	
Top-1	very	0.81	Top-2 (Sampling list)
Top-2	good	0.14	
Top-3	bad	0.02	
Top-4	wrong	0.002	
...	...	...	
Top-4899	was	0.0001	
Top-4900	will	0.0001	

Top-p

Select the k highest, then sample uniformly

The dog is

[Top-k]	[Token]	[Top-p (p: Probability)]	
Top-1	very	0.81	Cumulative probability exceeds 0.96 (0.81+0.14+0.02 > 0.96) (Sampling list)
Top-2	good	0.14	
Top-3	bad	0.02	
Top-4	wrong	0.002	
...	...	...	
Top-4899	was	0.0001	
Top-4900	will	0.0001	

Context limits & the cost of running a model

CONTEXT WINDOW

GPT-3.5	4 K tokens
Llama 3	8 K tokens
GPT-4o	128 K tokens

Longer than the window? → truncate or chunk →
motivates RAG.

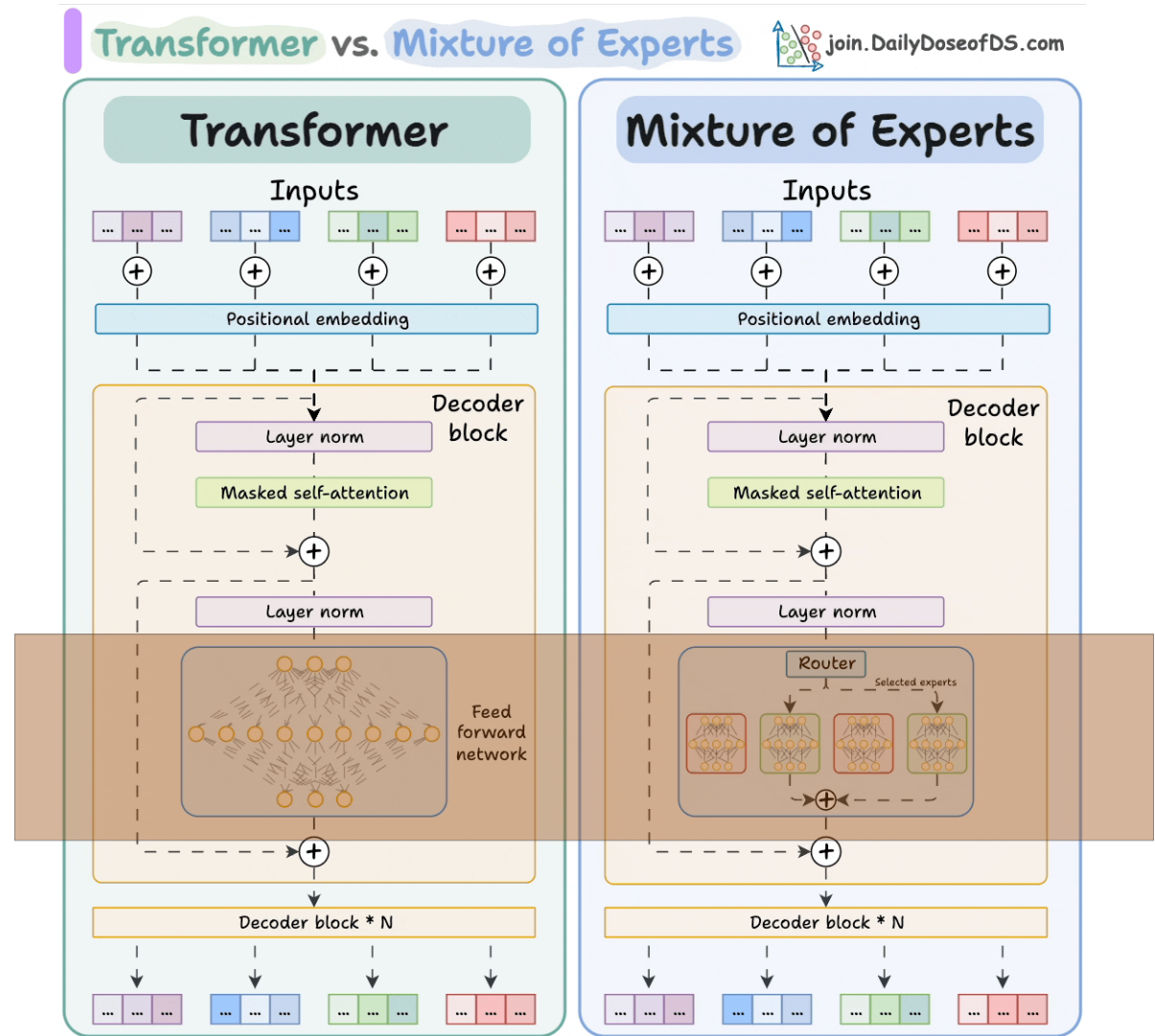
MEMORY COST · A 7B MODEL

float32	~28 GB
float16	~14 GB
int4 (quantised)	~3.5 GB

4-bit quantisation fits large models on a laptop —
Ollama does it automatically .

Mixture of Experts Models

Multi-layer perceptron (DNN) is replaced by many smaller networks



SECTION 6

Hallucinations

Why hallucinations happen

01

Distribution mismatch

Never shown every true fact → it **interpolates** the gaps.

02

No truth signal

Training rewards **fluency**, not accuracy.
A plausible wrong answer scores as well as a right one.

03

Probabilistic decoding

Temperature > 0 → the model **samples**; even a correct model errs by chance.

The model cannot distinguish **what it knows** from **what it infers**

LLM Hallucinations

Prompt:

“Who won the 1924 FIFA World Cup?”

✗ **Hallucinated answer:**

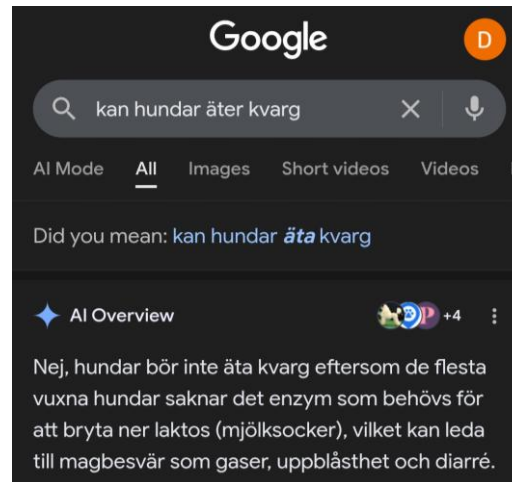
“Argentina won the 1924 FIFA World Cup.”

✓ **Reality:**

There **was no FIFA World Cup in 1924** — the first one was in **1930**.

Prompt:

“Can dogs eat quark?”



NO



YES

hallucinations are not bugs to be patched — they are a structural property

LLM Hallucinations

Here's What Happens When Your Lawyer Uses ChatGPT

A lawyer representing a man who sued an airline relied on artificial intelligence to help prepare a court filing. It did not go well.

 Share full article    1.1K



<https://www.nytimes.com/2023/05/27/nyregion/avianca-airline-lawsuit-chatgpt.html>

The lawsuit began like so many others: A man named Roberto Mata sued the airline Avianca, saying he was injured when a metal serving cart struck his knee during a flight to Kennedy International Airport in New York.

When Avianca asked a Manhattan federal judge to toss out the case, Mr. Mata's lawyers vehemently objected, submitting a 10-page brief that cited more than half a dozen relevant court decisions. There was *Martinez v. Delta Air Lines*, *Zicherman v. Korean Air Lines* and, of course, *Varghese v. China Southern Airlines*, with its learned discussion of federal law and "the tolling effect of the automatic stay on a statute of limitations."

There was just one hitch: No one — not the airline's lawyers, not even the judge himself — could find the decisions or the quotations cited and summarized in the brief.

That was because ChatGPT had invented everything.

The lawyer who created the brief, Steven A. Schwartz of the firm Levidow, Levidow & Oberman, threw himself on the mercy of the court on Thursday, saying in an affidavit that he had used the artificial intelligence program to do his legal research — "a source that has revealed itself to be unreliable."

Mr. Schwartz, who has practiced law in New York for three decades, told Judge P. Kevin Castel that he had no intent to deceive

Mitigation strategies — a ladder

Without fully retraining the LLM:

PARTIAL	Prompt engineering	“Cite sources. If unsure, say so.”
PARTIAL	Self-consistency / CoT	Sample several answers, pick the consistent one — helps reasoning
MAIN	RAG	Ground the model in retrieved context — the focus of this module
EXTENDED	LLM + structured knowledge	Hybrid with graphs → Module IV (GraphRAG)

SECTION 7

Retrieval Augmented Generation

The RAG idea: open-book, not closed-book

CLOSED-BOOK

Model must recall everything from training

→ gaps get filled by **hallucination**.

OPEN-BOOK · RAG

Retrieve facts at query time, inject into the prompt

→ the model accesses **real sources** within its prompt

LLMs are good at **text pattern matching** — retrieval handles **recall** of relevant information

Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS 2020 — the paper that named RAG.

RAG

R **Retrieve:** find useful information from your database.

A **Augment:** add the information to the prompt.

G **Generate:** create more accurate response.

Prompt:

“Who won the 1924 FIFA World Cup?”

✗ Hallucinated response:

“Argentina won the 1924 FIFA World Cup.”

Prompt:

“Who won the 1924 FIFA World Cup?”

Retrieve Relevant doc:

Wikipedia---history of FIFA world cup page

Augmented Prompt:

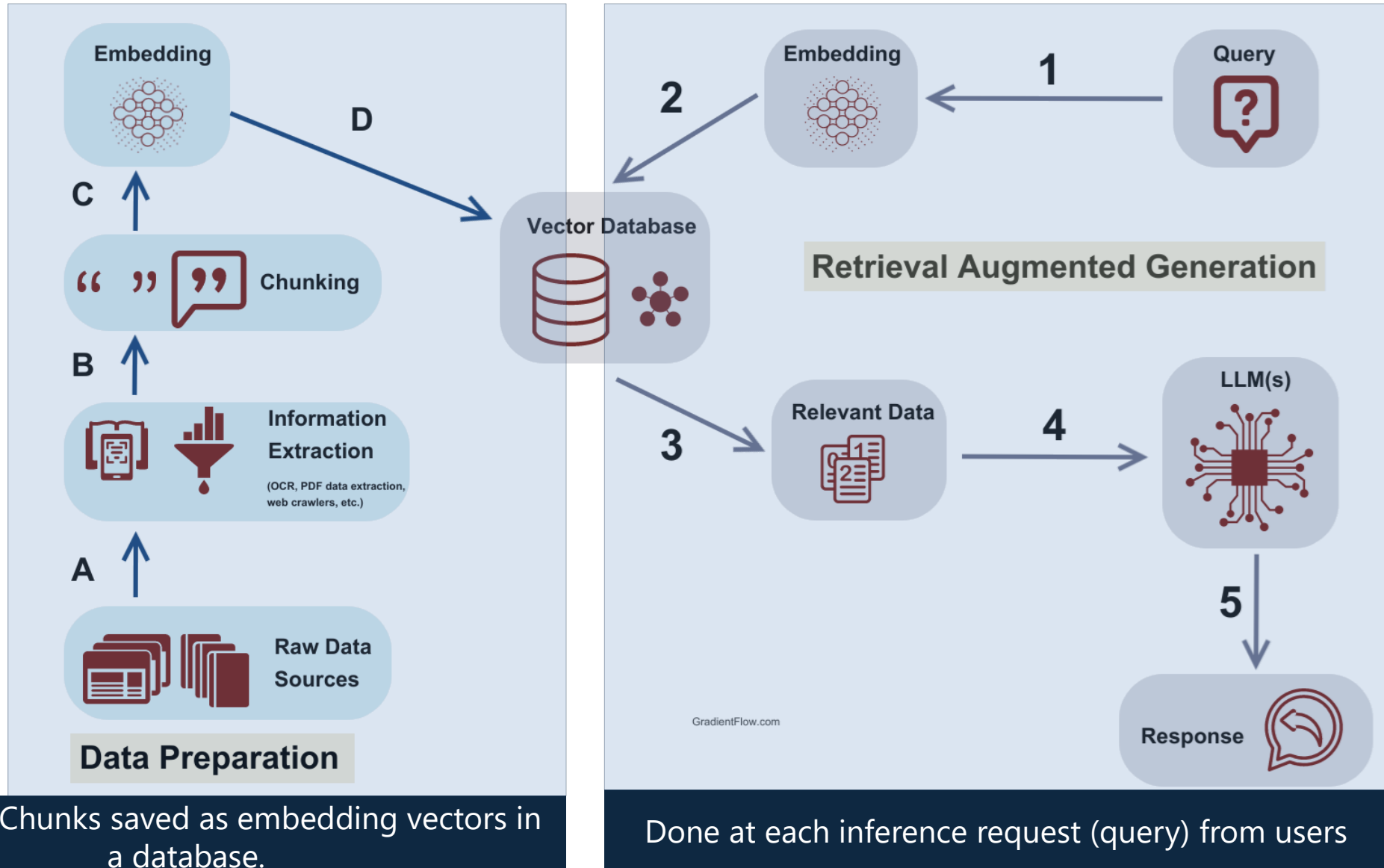
<context: The 1930 FIFA World Cup was the 1st FIFA World Cup, the world champ...>

“Who won the 1924 FIFA World Cup?”

✓ Response:

There was no FIFA World Cup in 1924 — the first one was in 1930.

RAG



RAG Architecture: step by step

Step 1 — Ingest: chunk documents → embed each chunk → store in FAISS

Step 2 — Query: embed the user question

Step 3 — Retrieve: find the top-k most similar chunks (cosine similarity)

Step 4 — Augment: inject retrieved chunks into the prompt

Step 5 — Generate: LLM reads augmented prompt → produces grounded answer

Tools used in the lab:

Embedding: sentence-transformers/all-MiniLM-L6-v2

Vector store: FAISS IndexFlatIP

LLM: Ollama (local) / SmolLM2 (Colab)

SECTION 8

Limits of RAG

The relationship problem

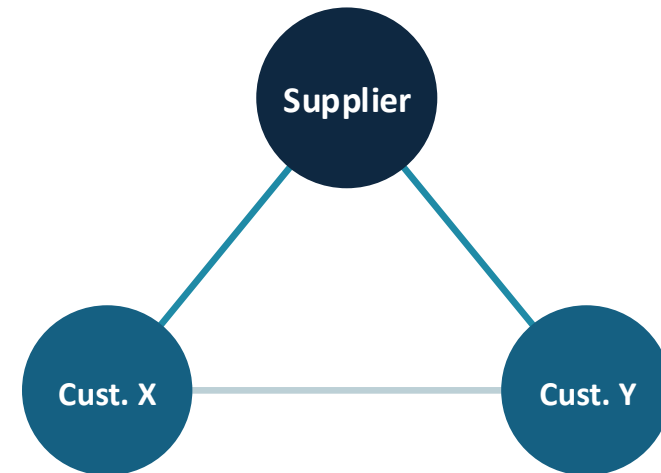
RAG returns a flat list

chunk: "Supplier A delivers to..."

chunk: "Customer X ordered..."

chunk: "Customer Y invoice..."

VS



No edges — the connection is invisible.

If your data has many relationships, you most likely need a graph.

Module II → Module III

WHAT WE COVERED

- Fundamentals of NLP and LLMs
- Transformer Architecture and Types of Transformers
- Stages to train an LLM
- Retrieval Augmented Generation

WHAT'S NEXT

Graph Neural Networks

Fundamentals of GNNs and their applicability

Daniel F. Pérez-Ramírez · daniel.perez.ra@gmail.com